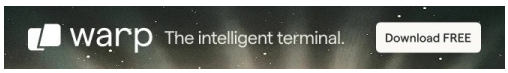
I Switched From Ollama And LM Studio To llama.cpp And Absolutely Loving It

Like Ollama, I can use a feature-rich CLI, plus Vulkan support in llama.cpp and it takes a lot less disk space, too.

Bhuwan Mishra
11 Oct 2025 · 6 min read · 7 comments

ON THIS PAGE

- My struggle with running local AI models
- Why not Ollama and LM Studio?
- Setting up Llama.cpp
  - Step 1: Download from GitHub
  - Step 2: Extract the zip file
  - Step 3: Add the Llama.cpp directory to the PATH environment variable
- llama.cpp: The best local AI stack for me
- llama.cpp for the win



My interest in running AI models locally started as a side project with part curiosity and part irritation with cloud limits. There's something satisfying about running everything on your own box. No API quotas, no censorship, no signups. That's what pulled me toward local inference.

My struggle with running local AI models

My setup, being an AMD GPU on Windows, turned out to be the worst combination for most local AI stacks.

The majority of AI stacks assume NVIDIA + CUDA, and if you don't have that, you're basically on your own. ROCm, AMD's so-called CUDA alternative, doesn't even work on Windows, and even on Linux, it's not straightforward. You end up stuck with CPU-only inference or inconsistent OpenCL backends that feel like a decade behind.

Why not Ollama and LM Studio?

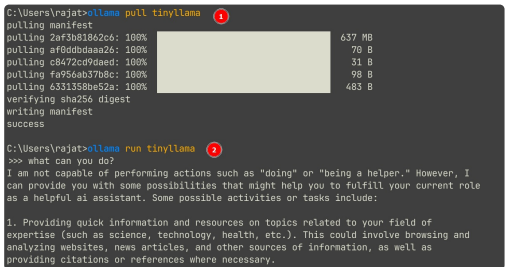
I started with the usual tools, i.e., Ollama and LM Studio. Both deserve credit for making local AI look plug-and-play. I tried LM Studio first. But soon after, I discovered how LM Studio hijacks my taskbar. I frequently jump from one application window to another using the mouse, and it was getting annoying for me. Another thing that annoyed me is its installer size of 528 MB.

I'm a big advocate for keeping things minimal yet functional. I'm a big admirer of a functional text editor that fits under 1 MB (Dred), a reactive JavaScript library and React alternative that fits under 1KB (Van JS), and a game engine that fits under 100 MB (Godot).

Then I tried Ollama. Being a CLI user (even on Windows), I was impressed with Ollama. I don't need to spin up an Electron JS application (LM Studio) to run an AI model locally.

With just two commands, you can run any AI models locally with Ollama.

```
ollma pull tinyllama
ollama run tinyllama
```



Featured

How Linux and BSD Distros Are Responding to the New Age Verification Laws

Firefox Is Getting a Major Redesign After 5 Years

Age Verification Laws Are Multiplying Like a Virus, and Your Linux Computer Might be Next

Why Linux Users Love to Hate Ubuntu

AI Can Now Easily Unmask Your Secret Online Life (Even If You Use a Fake Name)

Searcl

DOWNLOAD FOR FREE

FREE APP DOWNLOAD

FREE VPN F

Subscribe

DOWNLOAD FOR WINDOWS

FREE CELL PHONE LOCATOR

INSTALL WINDOWS 11

BEST DEALS ON SUVS

Latest

MidnightBSD Bans Users in Brazil and California, Warns More Regions Could

But once I started testing different AI models, I needed to reclaim disk space after that. My initial approach was to delete the model manually from File Explorer. I was a bit paranoid! But soon, I discovered these Ollama commands:

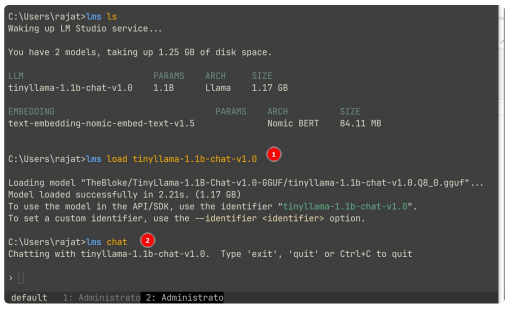
```
ollama rm tinyllama     #remove the model
ollama ls               #lists all models
```

Upon checking how lightweight Ollama is, it comes close to 4.6 GB on my Windows system. Although you can delete unnecessary files to make it slim (it comes bundled with all libraries like rocm, cuda_v13, and cuda_v12),

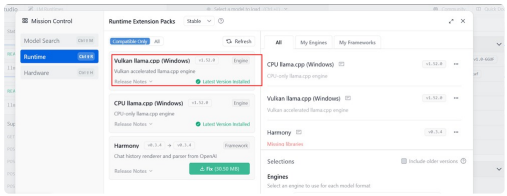
After trying Ollama, I was curious! Does LM Studio even provide a CLI? Upon my research, I came to know, yeah, it does offer a command lineinterface. I investigated further and found out that LM Studio uses Llama.cpp under the hood.

With these two commands, I can run LM Studio via CLI and chat to an AI model while staying in the terminal:

```
lms load <model name>   #Load the model
lms chat                #starts the interactive chat
```



I was generally satisfied with LM Studio CLI at this moment. Also, I noticed it came with Vulkan support out of the box. Now, I have been looking to add Vulkan support for Ollama. I discovered an approach to compile Ollama from source code and enable Vulkan support manually. That's a real hassle!



I just had three additional complaints at this moment. Every time I needed to use LM Studio CLI(lms), it would take some time to wake up its Windows service. LMS CLI is not feature-rich. It does not even provide a CLI way to delete a model. And the last one was how it takes two steps to load the model first and then chat.

After the chat is over, you need to manually unload the model. This mental model doesn't make sense to me.

That's where I started looking for something more open, something that actually respected the hardware I had. That's when I stumbled onto Llama.cpp, with its Vulkan backend and refreshingly simple approach.

Setting up Llama.cpp

Follow
09 Mar 2026

FRANK OS Turns a Microcontroller Into a Tiny Retro Desktop PC
07 Mar 2026

FOSS Weekly #26.10: Age Verification in Linux, systemd Troubleshooting Tools, Graphene Phone, Longer Linux LTS Kernels and More
05 Mar 2026

Become a Better Linux User

With the FOSS Weekly Newsletter, you learn useful Linux tips, discover applications, explore new distros and stay updated with the latest from Linux world

Your email address

Subscribe

Discover more

NVIDIA + CUDA
ollama
SYCL

The tutorial was performed on Windows because that's the system I am using currently. I understand that most folks here on It's FOSS are Linux users and I am committing blasphemy of sort but I just wanted to share the knowledge and experience I gained with my local AI setup. You could actually try similar setup on Linux, too. Just use Linux equivalent paths and commands.

Step 1: Download from GitHub

Head over to its GitHub releases page and download its latest releases for your platform.

If you'll be using Vulkan support, remember to download assets suffixed with **vulkan-x64.zip** like llama-b6710-bin-ubuntu-vulkan-x64.zip, llama-b6710-bin-win-vulkan-x64.zip.

Step 2: Extract the zip file

Extract the downloaded zip file and, optionally, move the directory where you usually keep your binaries, like `/usr/local/bin` on macOS and Linux. On Windows 10, I usually keep it under `%USERPROFILE%\.local/bin`.

Step 3: Add the Llama.cpp directory to the PATH environment variable

Now, you need to add its directory location to the PATH environment variable.

On Linux and macOS (replace path-to-llama-cpp-directory with your exact directory location):

```
export PATH=$PATH:"<path-to-llama-cpp-directory>"
```

On Windows 10 and Windows 11:

```
setx PATH=%PATH%;:"<path-to-llama-cpp-directory>"
```

Now, Llama.cpp is ready to use.

llama.cpp: The best local AI stack for me

Just grab a .gguf file, point to it, and run. It reminded me why I love tinkering on Linux in the first place: fewer black boxes, more freedom to make things work your way.

With just one command, you can start a chat session with Llama.cpp:

```
llama-cli.exe -m e:\models\Qwen3-8B-Q4_K_M.gguf --interactive
```

If you carefully read its verbose message, it clearly shows signs of GPU being utilized:

With llama-server, you can even download AI models from Hugging Face, like:

```
llama-server -hf itlwas/Phi-4-mini-instruct-Q4_K_M-GGUF:Q4_K_M
```

`-hf` flag tells to download the model from the Hugging Face repository.

You even get a web UI with Llama.cpp. Like run the model with this command:

```
llama-server -m e:\models\Qwen3-8B-Q4_K_M.gguf --port 8080 --hos
```

This starts a web UI on http://127.0.0.1:8080, along with the ability to send an API request from another application to Llama.

Let's send an API request via curl:

```
curl http://127.0.0.1:8080/completion -H "Content-Type: applicat
```

- temperature controls the creativity of the model's output
- max_tokens controls whether the output will be short and concise or a paragraph-length explanation.

llama.cpp for the win

What am I losing by using llama? Nothing. Like Ollama, I can use a feature-rich CLI, plus Vulkan support. All comes under 90 MB on my Windows 10 system.

Now, I don't see the point of using Ollama and LM Studio, I can directly download any model with llama-server, run the model directly with llama-cli, and even interact with its web UI and API requests.

I'm hoping to do some benchmarking on how performant AI inference on Vulkan is as compared to pure CPU and SYCL

implementation in some future post. Until then, keep exploring AI tools and the ecosystem to make your life easier. Use AI to your advantage rather than going on endless debate with questions like, will AI take our jobs?

AI

Share Share Share Share Email Feedback

ABOUT THE AUTHOR

Bhuwan Mishra

Enter your comment here. Please note that hateful text, abuses and NSFW words won't be allowed. Be resepectful, even if you disagree with the article or comments. Also note that all comments are manually approved. Wait for the moderators to approve your

Submit Reply

B I U S GIF

Sign in to your member account

Commenting experience is a lot better for the members. You can vote, receive notifications for replies, earn karma points and badges for your commenting activities. Highly recommended that you sign in with your account to leave a comment. Otherwise, you can also comment as a guest without all the fun.

Your Email Address (Won't be published here)

Username

You may receive notification if you receive a reply to your message

Submit

7 comments Newest First

jonnylbluejeans 5 months ago

I went down almost the exact same path as you did. Did you get a MultiModal LLM working on llama.cpp? I am having trouble with that

Reply

MEMBER **Martian Bold** 5 months ago

Excuse the newbie question, but what is the minimum hardware configuration to run a test with llama-cpp? Is a dedicated graphics card required? How much RAM?

Reply

5 months ago

MEMBER **Samuel**

thanks for this article, very good. I see there are several download options to llama.cpp. Will we, users, always look to use the last one as recommendation?

Reply

sameerhere 5 months ago

Awesome... will try out

Reply

Sasori7 5 months ago

This article is ironic.
'I hate installing 400mb apps; I downloaded 40gb llm. '
'so, I'll jump through 50 hoops to blah blah blander blander'

Reply

READ NEXT

AI Can Now Easily Unmask Your Secret Online Life (Even If You Use a Fake Name)

OpenClaw Alternatives That You Can Run on Raspberry Pi Like Devices

VibeOps Folks Rejoice! Warp Launches Oz for Managing AI Coding Agents in the Cloud

I Ditched Claude Code and Now Using Open Source Qwen AI for Real Sysadmin Work

I Used This Open Source Library to Integrate OpenAI, Claude, Gemini to Websites Without API Keys

Load More (2) | View All (7)

FastComments.com

Become a Better Linux User

With the FOSS Weekly Newsletter, you learn useful Linux tips, discover applications, explore new distros and stay updated with the latest from Linux world

Your email address SUBSCRIBE

©2026 It's FOSS. Hosted on Digital Ocean (get $200 in FREE credits) & Published with Ghost & Rinne. System